HMI · MES 설비 운영 시스템

가상 포장라인을 실제 암호화 OPC UA로 연결해, 설비 감시·제어부터 작업지시·실적 정산까지 잇는 웹 HMI·MES

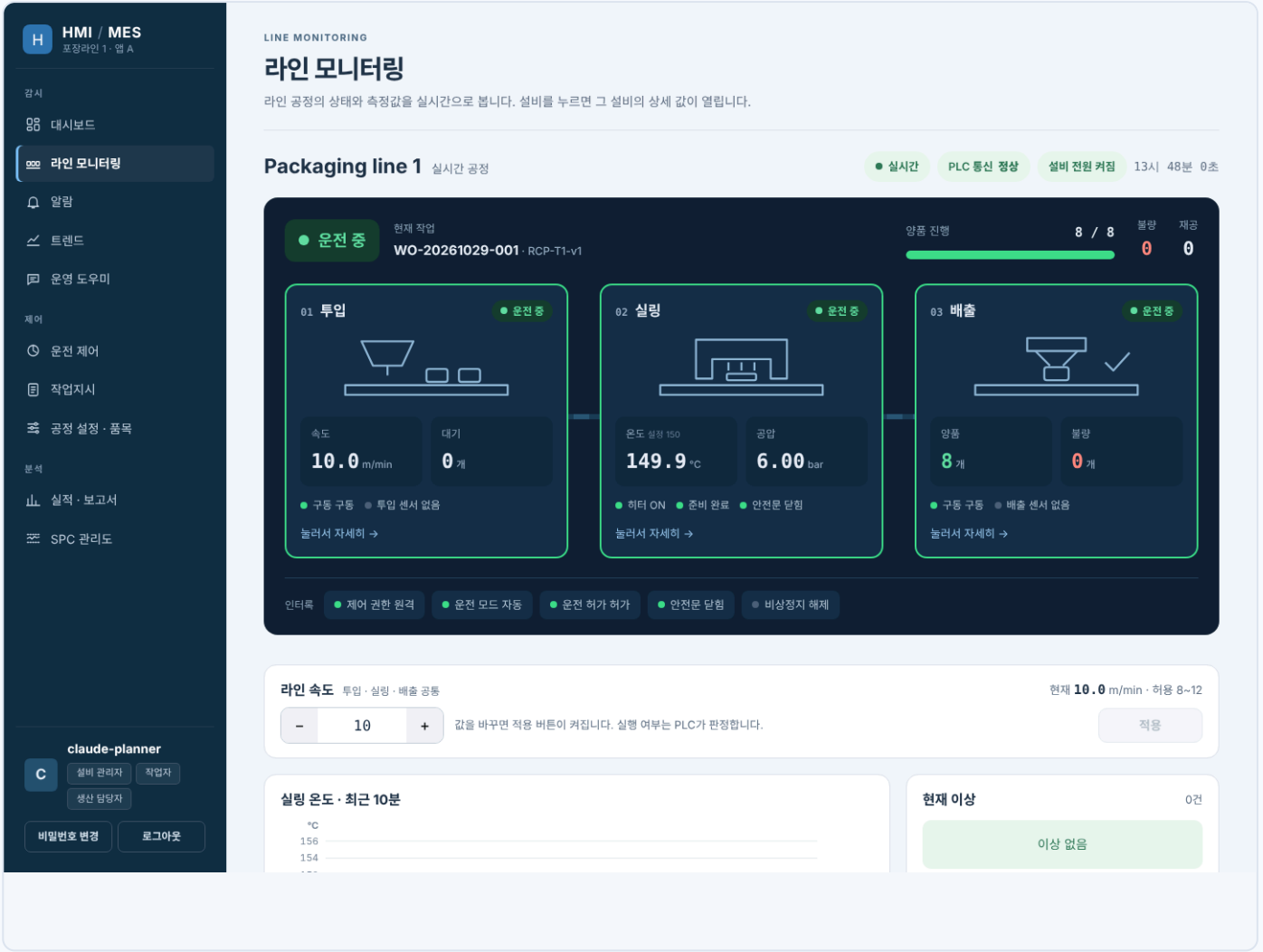
HMI
MES.

버튼을 누른 것과
설비가 실제로 움직인 것을
끝까지 구분하는 시스템.

Python · FastAPI | PostgreSQL | OPC UA · asyncua | React · TypeScript | Vercel · Neon

작성자 전우석

개발 기간 2026.10.20 ~ 10.28 (9일) · 커밋 312개
기획 방향·운영 규칙 결정, 화면 검수, 개발 조율
Codex · Claude Code를 활용한 구현과 검증



공개 배포 hmi-mes.vercel.app · 2026-10-29 실제 운전 중인 라인 모니터링 화면

설비가 실제로 한 일만 실적이 됩니다

투입 컨베이어 → 실링기 → 검사 배출기로 이어지는 포장라인 1개를 감시·제어하고, 작업지시별 생산 실적을 정산합니다.

무엇을 하는 시스템인가

설비 시뮬레이터(앱 B)가 세 설비와 가상 PLC를 돌리고, 운영 시스템(앱 A)이 OPC UA로 값을 수집해 웹 화면으로 보여 줍니다. 작업자는 화면에서 작업지시를 불러와 운전하고, 시스템은 생산 사건을 원래 작업 실행에 귀속해 실적을 정산합니다. 두 앱은 DB와 메모리를 공유하지 않습니다.

핵심 규칙 3가지

1. **판정은 PLC가 합니다**
   화면은 요청만 보냅니다. 응답이 끊기면 같은 명령 ID로 결과만 조회하고, 새 ID로 다시 실행하지 않습니다.

2. **모르는 것은 모른다고 표시합니다**
   통신 단절·품질 불명·누락을 0이나 정상값, 정산 완료로 바꾸지 않습니다.

3. **정산이 끝나야 다음 작업입니다**
   생산 사건은 고유 ID로 한 번만 반영하고, 설비 최종 수량과 대조한 뒤에만 다음 작업을 연결합니다.

규모 (저장소 실측)

62 DB 테이블	**21** 트리거로 수정·삭제를 막는 테이블	**25** DB 함수 (트리거 48개)
28 마이그레이션	**10** 메뉴 화면 (로그인·설비 상세 창 별도)	**2,354** Python 자동화 시험 (+ 화면 91)

테이블·함수·트리거 수는 로컬 PostgreSQL에 마이그레이션 0028_work_order_reuse까지 적용해 센 값입니다. 저장 프로시저 대신 트랜잭션은 Python 코드에서 제어하고, DB 함수는 불변 이력 보호와 보관 정리에 씁니다. 시험 결과는 12쪽에 있습니다.

두 앱은 OPC UA로만 만나고, 화면과 게이트웨이는 API만 거칩니다

KULS 공개 HMI 구성예시(현장설비 → Master PLC → OPC 서버 → HMI·MES 서버 → 모니터링)의 각 계층을 직접 구현



앱 A · 운영 시스템

인증, 명령 원장, 수집, 작업지시, 정산, 분석을 맡습니다. 분석 조회는 별도 함수로 분리해 무거운 조회가 제어 API를 막지 않게 했습니다.

앱 B · 설비 시뮬레이터

운전 조건 판단과 실행 결과의 최종 판정을 맡습니다. 운영 시스템이 멈춰도 자체 규칙으로 동작합니다.

게이트웨이

OPC UA 값을 읽어 묶음으로 보내고, 명령을 PLC에 전달합니다. 응답을 받기 전에는 버퍼를 지우지 않습니다.

> 게이트웨이와 브라우저는 DB에 직접 붙지 않습니다. 설비 쪽은 상시 실행이 아니며, 화면의 전원 요청이 있을 때만 켜집니다.

계층마다 “실패했을 때 무엇이 남는가”를 기준으로 골랐습니다

선택 이유와 함께, 그 선택으로 감수한 점을 같이 적었습니다.

계층	선택	고른 이유	감수한 점
설비 연동	OPC UA (asyncua) Basic256Sha256	실제 산업 현장 표준이고, 서명·암호화와 인증서 검증을 프로토콜 수준에서 제공합니다. 모의 HTTP가 아니라 실제 프로토콜로 연결했습니다.	세션·인증서 관리가 복잡해 연결 실패 경로의 시험이 많아졌습니다.
설비 앱 저장	SQLite	물품·수량·사건·명령을 한 파일, 한 트랜잭션에 원자적으로 저장합니다. 프로세스가 중간에 죽어도 일부만 저장되지 않습니다.	단일 프로세스 쓰기 전제입니다.
게이트웨이 버퍼	SQLite 디스크 큐	통신이 끊겨도 디스크에 쌓아 두고, DB 커밋 응답을 받은 묶음만 지웁니다. 30분·100MB 한도와 누락 구간을 기록합니다.	한도를 넘는 단절에서는 오래된 관측값을 잃고, 그 사실을 누락으로 남깁니다.
API	Python · FastAPI Pydantic · SQLAlchemy Core	계약 모델(Pydantic)로 경계에서 형식을 다시 검증하고, ORM 자동 동작 없이 트랜잭션과 잠금 순서를 직접 제어합니다.	쿼리를 직접 쓰는 만큼 코드량이 늘었습니다.
운영 DB	PostgreSQL · Alembic	권고 잠금(advisory lock), 행 잠금, 부분 고유 인덱스, 트리거로 불변 이력을 DB가 직접 지킵니다. 스키마는 마이그레이션 28개로 관리합니다.	무료 Neon 저장 한도 안에서 보관 기간과 압축을 별도로 설계해야 했습니다.
화면	React · TypeScript · Vite	서버 계약을 타입으로 맞추고, 화면 갱신을 웹소켓과 폴링 중에서 배포 환경에 맞게 고릅니다.	서버리스 배포에서는 폴링을 써서 갱신 간격만큼 늦게 보입니다.
시험	pytest · Docker PostgreSQL Playwright	실제 PostgreSQL과 실제 브라우저로 경로 전체를 시험합니다. CI 대신 저장소의 로컬 검사로 검증합니다.	전체 검사에 20분 이상 걸립니다.
배포	Vercel · Neon (무료)	비용 0으로 공개 데모를 운영합니다. 함수를 화면·수집·분석 3개로 나눠 각각의 시간 제한 안에 둡니다.	DB 사용량 한도가 있고, 설비 쪽은 상시 연결이 아닙니다.

테이블을 수명에 따라 네 묶음으로 나눴습니다

기준정보는 버전으로, 현재 상태는 잠금으로, 이력은 트리거로, 운영 기록은 보관 정책으로 관리합니다.

기준정보

바꾸지 않고 새 버전을 만듭니다

sites · lines
공장 · 라인

assets
설비 3대
→ lines

tag_definitions
태그 48개
→ assets

item_master
품목
→ users

recipe_versions
공정 설정 버전
→ item_master

users · roles
계정 · 역할 4종

현재 상태

행 잠금 아래에서만 바뀝니다

work_orders
작업지시
→ recipe_versions · lines

work_order_execution_links
작업지시 ↔ 실행 회차
→ work_orders · production_executions

production_executions
작업 실행
→ source_streams

execution_totals
실행별 집계
→ production_executions

command_requests
명령 원장
→ production_executions

line_queue_entries
작업 대기열
→ work_orders

stream_progress
처리 커서
→ source_streams

지울 수 없는 이력

트리거가 수정·삭제를 거절합니다

item_audit
품목 변경 기록
→ item_master

registration_receipts
등록 영수증
→ recipe_versions · work_orders

work_order_cancellations
작업지시 취소
→ work_orders

line_queue_audit
대기열 변경
→ lines

alarm_actions
알람 확인·조치
→ alarm_instances

downtime_reason_history
정지 사유 이력
→ users

capture_clock_evidence
수집 시각 근거
→ ingest_batches

운영 기록

보관 기간이 지나면 압축·정리합니다

event_inbox
생산 사건 원본
→ source_streams

production_events
실적 반영 사건
→ event_inbox · production_executions

ingest_batches
수집 묶음 원문
→ source_streams

stream_gaps
누락 구간
→ source_streams

analog_history_raw · minutes
1초 · 1분 트렌드
→ source_streams

auth_sessions · auth_audit
세션 · 인증 기록
→ users

전체 62개 테이블 중 업무 흐름의 중심 26개입니다. → 는 외래키 참조 대상입니다. 트리거로 수정·삭제를 막는 테이블은 모두 21개이며, 생산 사건 원본(event_inbox)은 트리거 대신 사건 ID 기본키와 애플리케이션 규칙으로 한 번만 쓰입니다.

수집 요청 하나는 하나의 트랜잭션 안에서 끝납니다

IngestionService.ingest — 게이트웨이가 보낸 수집 묶음 한 건의 처리 순서. 중간에 실패하면 전부 되돌리고 ACK를 보내지 않습니다.

1 인증 범위 확인
토큰으로 확인한 게이트웨이·수집원·스트림이 묶음 본문과 같은지 봅니다. 다르면 거절합니다.

2 트랜잭션 시작 · 잠금
게이트웨이 권고 잠금 → 라인·스트림 행 잠금 → 사건 ID별 권고 잠금(정렬 순서, 교착 방지).

3 같은 묶음인지 판별
같은 묶음 ID가 이미 있으면 내용 해시를 비교합니다.

- **내용까지 같음** — 저장해 둔 원래 ACK를 그대로 돌려줍니다. 다시 반영하지 않습니다.
- **내용이 다름** — 충돌 증거를 저장하고 스트림을 막습니다. ACK는 주지 않습니다.

4 사건별 수신
이미 받은 사건 ID면 원래 영수증만 돌려주고, 처음이면 원본을 먼저 저장한 뒤 실적에 반영합니다.

5 처리 커서 · 누락 구간
순번이 이어지면 커서를 전진시키고, 빈 순번은 누락 구간으로 기록합니다.

6 ACK 생성 · 묶음 원문 저장
ACK를 만들어 스스로 검증한 뒤 묶음 원문과 함께 저장합니다.

7 시각 근거 · 알람 · 트렌드
같은 트랜잭션 안에서 파생 기록을 만듭니다.

8 커밋 → ACK 반환
커밋이 끝난 뒤에만 ACK를 보냅니다. 게이트웨이는 이 ACK를 받아야 버퍼를 지웁니다.

왜 한 트랜잭션인가

중복 제거, 실적, 처리 커서가 따로 저장되면 재시작 때 실적이 두 번 잡히거나 커서만 앞서 나갑니다. 셋을 같이 커밋하거나 같이 버립니다.

재전송은 안전하다

응답이 끊겨 게이트웨이가 같은 묶음을 다시 보내도 3단계에서 원래 ACK를 돌려받습니다.

충돌은 숨기지 않는다

같은 ID에 다른 내용이 오면 덮어쓰지 않고 증거를 남긴 채 그 스트림을 멈춥니다.

같은 사건은 원래 영수증으로 답하고, 처음 온 사건만 반영합니다

실제 코드 발췌 · apps/api/ingestion.py · 고정 커밋 8134cb8

L976~L985, L779~L810 발췌 · 보관 기간·세대 검사와 저장 열 일부 생략

```
# ingest(): 트랜잭션과 잠금 순서
with self._transaction() as connection:
    self._advisory(connection, "gateway", body.gateway_id)
    source = self._lock_stream(connection, body.source_id, body.stream_id)
    if source["gateway_id"] != body.gateway_id:
        raise SourceBindingError("Gateway is not explicitly registered")
    for event_id in sorted(event.event_id for event in body.events):
        self._advisory(connection, "event", event_id)

# _receive_event(): 사건 단위 중복 제거
existing = (
    connection.execute(
        sa.select(event_inbox).where(
            event_inbox.c.event_id == event.event_id,
        )
    )
    .mappings()
    .first()
)
if existing is not None:
    return self._receipt(existing)
payload = event.model_dump(mode="json")
connection.execute(
    event_inbox.insert().values(
        event_id=event.event_id,
        execution_id=event.execution_id,
        applied_at=None,
        reason_code="APPLICATION_PENDING",
    )
)
reason = self._apply_event(connection, event, now)
self._set_application(connection, event.event_id, reason, now)
return EventReceipt(
```

수집 처리 코드 ↗ · 수집·ACK 계약 ↗

① 잠금 순서를 고정

게이트웨이 → 스트림 행 → 사건 ID 순으로 잠급니다. 사건 ID는 정렬해서 잠가, 두 묶음이 같은 사건을 반대 순서로 잡는 교착을 막습니다.

② 이미 받은 사건은 영수증만

사건 ID로 원본을 찾으면 다시 반영하지 않고 그때의 결과를 그대로 돌려줍니다.

③ 원본 먼저, 반영은 그다음

원본을 “반영 대기”로 저장한 뒤 실적에 반영합니다. 반영이 거절돼도 원본과 사유는 남습니다.

④ 실패는 격리로 기록

반영할 수 없는 사건은 지우지 않고 격리(QUARANTINED)로 표시해, 나중에 원래 실행으로 다시 처리할 수 있게 합니다.

권한은 첫 제어 경로부터, 화면이 아니라 서버가 검사합니다

역할 4종 × 권한 9종 · apps/api/auth.py의 PERMISSIONS 정의 그대로

권한	조회자	작업자	생산 담당자	설비 관리자
조회 `read`	●	●	●	●
시작·일시정지 등 운전 `operate`		●		
작업 불러오기 `load_job`		●	●	
작업 종료 `end_run`		●	●	
알람 확인·조치 `alarm_action`		●		
정지 사유 입력 `downtime_annotate`		●	●	
작업지시 등록·배포 `production_manage`			●	
공정 설정 관리 `parameters_manage`			●	
설비 관리 `equipment_manage`				●

로그인 계정 하나에 여러 역할을 줄 수 있습니다. 게스트 둘러보기는 조회자 권한만 있고, 쓰기 요청은 403으로 거절됩니다.

세션

비밀번호는 scrypt(N=2^{15})로 저장합니다. 세션은 30분 동안 쓰지 않으면, 또는 로그인 후 8시간이 지나면 끝납니다. 초기 비밀번호는 바꾸기 전까지 다른 작업을 막습니다.

변경 요청

모든 쓰기 요청에 CSRF 토큰과 정확한 Origin 일치를 요구합니다. 로그인·인증 결과는 auth_audit에 남깁니다.

설비 연동

게이트웨이는 별도 서비스 토큰으로 수집 API에 들어오고, 인증된 게이트웨이·수집원과 묶음 본문이 다르면 거절됩니다. OPC UA는 서명·암호화와 인증서 검증을 거칩니다.

비밀값

DB 접속정보·토큰·비밀번호는 소스·문서·로그에 쓰지 않고 배포 환경 변수로만 넣습니다.

동시에 들어와도, 다시 들어와도, 중간에 죽어도 숫자가 맞아야 합니다

문제 상황마다 어떤 장치로 막았는지 정리했습니다.

상황	막는 장치	결과
같은 수집 묶음이 두 번 도착	묶음 ID + 내용 해시 비교, 사건 ID 기본키	원래 ACK·영수증을 돌려주고 실적은 한 번만 반영
두 묶음이 같은 사건을 동시에 처리	사건 ID별 트랜잭션 권고 잠금, 정렬된 잠금 순서	한쪽이 끝날 때까지 기다림, 교착 없음
명령 응답이 중간에 끊김	명령 원장의 원래 command_id, 만료 시 결과 불명 예약	새 ID로 재실행하지 않고 원래 ID로 결과만 조회
같은 작업지시를 두 번 대기열에 넣음	대기 중(WAITING)인 항목에만 거는 부분 고유 인덱스	대기 중 중복은 DB가 거절, 끝난 회차는 다시 넣을 수 있음
설비 앱이 배출 확정 중에 죽음	물품·수량·사건을 SQLite 한 트랜잭션에 저장	전부 남거나 전부 없음, 반쯤 저장된 수량 없음
이력 행을 실수로 고치거나 지움	UPDATE·DELETE를 거절하는 트리거 (21개 테이블)	DB 수준에서 거절, 애플리케이션 버그로도 바꿀 수 없음
마이그레이션이 두 곳에서 동시에 실행	스키마 전용 권고 잠금	한 번에 하나만 적용
작업 전환 때 이전 수량이 섞임	재공 0 확인과 설비 최종 요약 대조 후에만 다음 작업 연결	실행마다 수량이 분리, 불일치는 정산 보류

사람이 반복하던 확인을 시스템이 대신합니다

설비 쪽 자동화와 화면 쪽 동작 방식을 함께 정리했습니다.

자동화

원격 전원

화면의 전원 요청을 런타임 에이전트가 받아 설비 앱·게이트웨이를 켜고, 60분 뒤 스스로 끕니다. 비용과 DB 사용량을 묶어 두기 위해서입니다.

자동 시작 · 자동 종료

예열이 끝나면 시작, 목표 수량에 도달하면 종료를 선택할 수 있습니다. 둘 다 PLC 판정을 거칩니다.

재전송

게이트웨이는 ACK를 못 받은 묶음을 자동으로 다시 보냅니다. 6쪽의 멱등 처리 덕분에 안전합니다.

입력 자동화

품목을 고르면 공정 설정 값을, 공정 설정을 고르면 작업지시 번호·품목을 채웁니다.

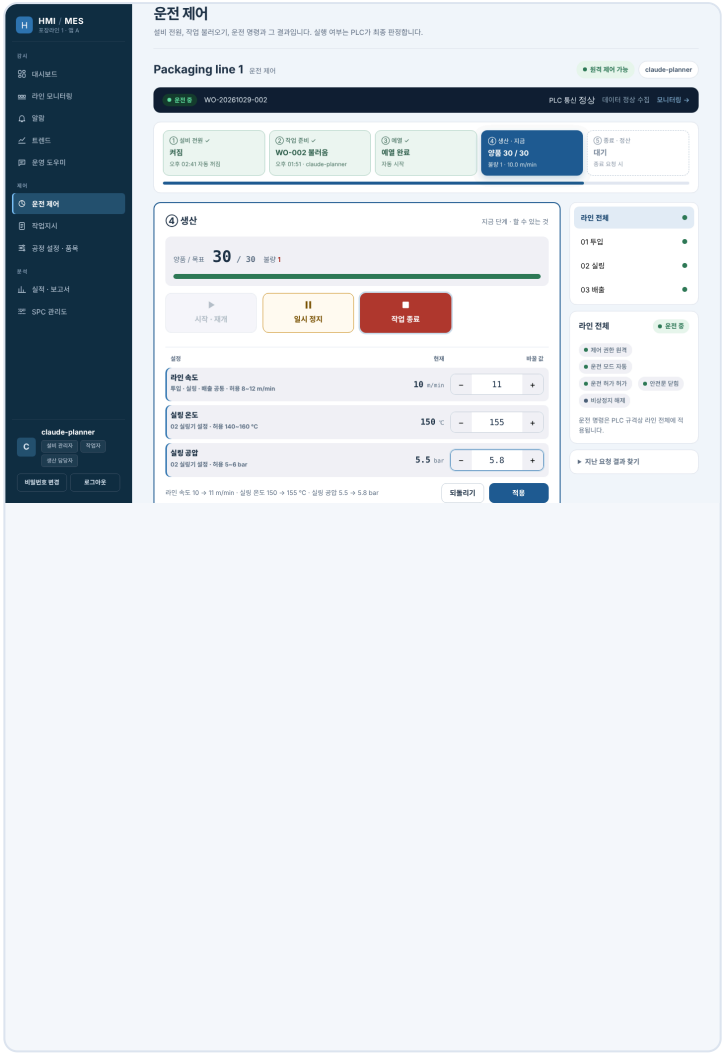
프론트엔드

갱신 방식

배포 환경에 따라 웹소켓 또는 폴링을 고릅니다. 탭이 보이지 않으면 폴링을 멈춰 DB 사용량을 줄입니다.

상태 표시

값마다 품질을 함께 받아 “불명”을 정상과 다른 색으로 표시합니다. 명령은 요청 저장 → PLC 접수 → 실행 완료의 단계로 보여 줍니다.



운전 중 설정 변경. 바꿀 값·허용 범위·변경 내용을 적용 전에 보여 줍니다.

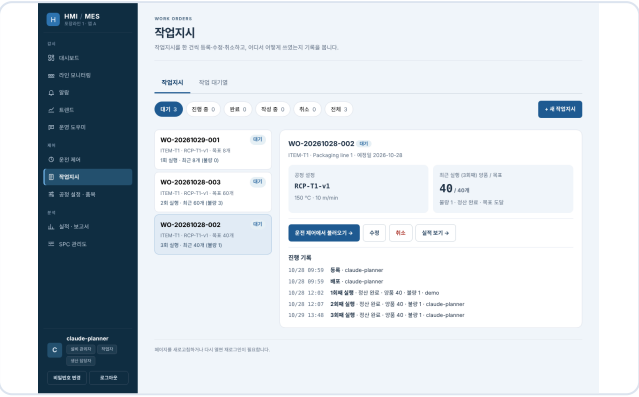


휴대폰 폭(390px) 운전 제어 화면

작업지시 등록부터 정산 · 다음 작업까지 한 흐름으로 이어집니다

2026-10-29 공개 배포를 실제로 운전하며 촬영 · 작업 실행 3건 모두 정산 완료

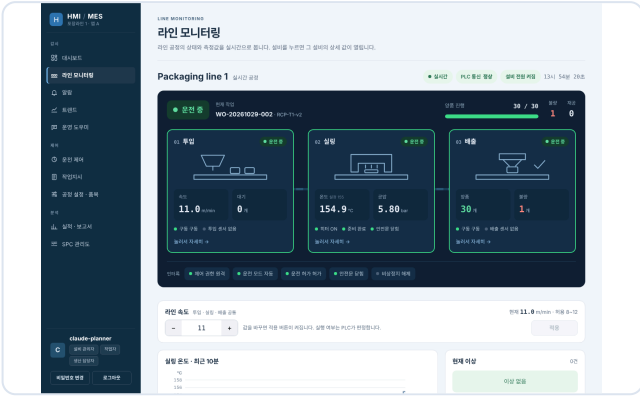
등록·배포	전원	불러오기	예열	생산	종료	정산	다음 작업
작업지시	설비 기동	설비에 연결	PLC 판정	설정 변경	재공 정리	수량 대조	재실행 가능



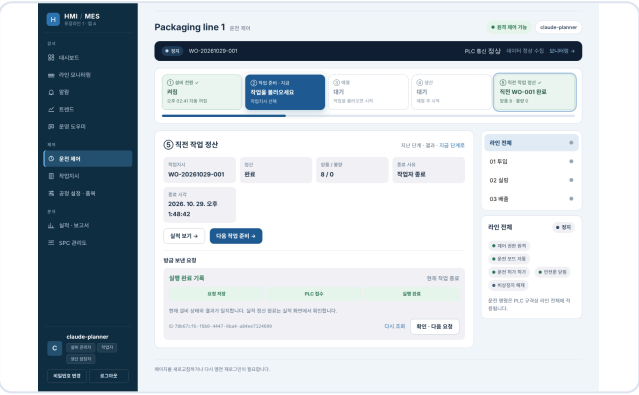
작업지시 · 같은 지시서를 3회 실행한 기록



운전 제어 · 단계 바와 현재 단계



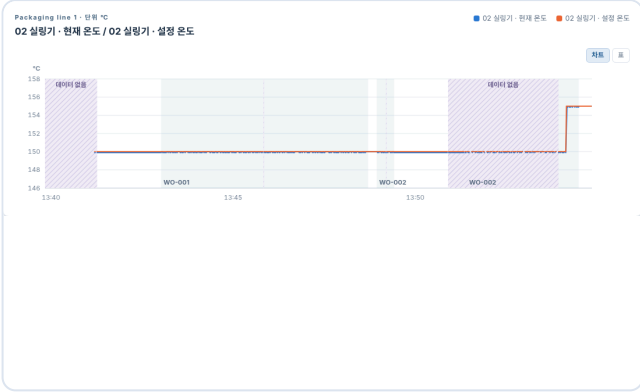
라인 모니터링 · 세 설비와 라인 속도



정산 · 직전 작업 양품 8 · 불량 0



보고서 · 회차별 실적과 가동 시간



트렌드 · 설정 150 → 155 °C 변화

단위에서 실제 경로까지, 실제 DB와 실제 브라우저로 시험했습니다

커밋 c8a24c1 기준 로컬 전체 검사 (2026-10-23)

층	무엇을 확인하나	도구
단위	PLC 판정, 상태 전이, 정산 계산, 권한 표	pytest
계약	태그 48개, 명령·사건·전송 형식, 직렬화	pytest · Pydantic
통합	실제 PostgreSQL에서 수집·중복·잠금·트리거·마이그레이션	Docker PostgreSQL
종단	설비 앱 → OPC UA → 게이트웨이 → API → DB → 브라우저	Playwright · Chromium
화면	화면 상태 계산, 입력 검증	Node 시험
장시간·성능	30분 단절 복구, 7일 이력 조회	전용 실행기 (과거 로컬)

검증 원칙

미실행·실패·차단을 통과로 적지 않습니다. 시험 개수는 자동화 시험의 실행 수이며 작성량이 아닙니다.

2,354
Python 통과 / 2,354

0
실패 · 오류

91
화면 시험 통과

기준 커밋 이후의 변경
이 결과는 2026-10-23 커밋 c8a24c1의 전체 로컬 검사입니다. 이후 화면 개편과 품목 기준정보 도입에 맞춰 종단 시험 일부를 갱신하는 중이며, 최신 버전의 전체 통과는 아직 다시 확인하지 않았습니다.

과거 기록	결과
30분 통신 단절 복구 (CX-19)	과거 로컬 구성에서 1,800초 단절 후 208.3초 만에 회복, 통과. 앞선 두 번은 실패
7일 이력 조회 (CX-20)	합성 7일 이력·RAM 디스크에서 사용자 10명 p95 1.34~1.40초, 통과

과거 기록 두 건은 최신 공개 배포에서 다시 검증하지 않았습니다.

구현은 AI에게 맡기고, 결정과 검증의 기준은 사람이 쥐었습니다

개발 세션과 검증 세션을 나누고, 공개 배포는 사용자의 지시가 있을 때만 했습니다.

사용자 기획 방향 · 운영 규칙 결정 → **Claude Code** 작업지시서 23건 작성 · 계약 조정 → **Codex** 서버 구현 · 작업 보고서 43건 → **Claude Code** 독립 검증 · 화면 · 통합 → **사용자** 화면 사용 · 배포 지시 → **main 병합** Vercel 자동 배포

직접 써 보며 찾은 결함

사용자가 발견한 것	바뀐 것
작업할 때마다 작업지시를 새로 만들어야 했다	작업지시를 유지하고 회차로 재실행 (마이그레이션 0028)
제어 화면인데 온도·공압을 못 바꿨다	공정 설정에 공압 추가, 운전 중 허용 폭 검사 (0027)
알람 보기 버튼이 빈 목록으로 갔다	해당 설비 창·운전 제어로 연결
불러오기를 눌렀는데 버튼이 또 있었다	불러오기를 한 번 클릭으로 정리
트렌드가 어느 설비의 값인지 몰랐다	설비·태그 이름과 단위별 차트로 분리

사용자가 정한 것

비용과 데이터

유료 요금제를 쓰지 않는다. 공개 DB를 쓰기 전에는 반드시 확인을 받는다.

범위

8시간 연속 운전은 이번 범위에서 뺀다. 같은 작업지시는 유지하고 다시 실행한다.

복구 판단

시험 데이터 정리로 수집 순번이 깨졌을 때, 원본을 지어내지 않고 스트림을 초기화하는 쪽을 승인했다.

본인 기여: 기획 방향·운영 규칙 결정, 화면 검수와 결함 발견, 범위와 복구 방식 결정, 개발 조율. **AI 지원:** 코드 작성, 시험 생성, 문서화. 모든 코드를 직접 작성했다고 표현하지 않습니다.

하지 않은 것을 먼저 밝힙니다

시뮬레이터 기반 프로토타입이며, 상용 현장 운영 성과로 표시하지 않습니다.

하지 않은 것

항목	현재 상태
실물 장비 연동	실물 PLC·센서 없음. 설비와 PLC는 소프트웨어 시뮬레이터입니다.
공개 배포 성능 측정	단절 복구·이력 조회 성능은 과거 로컬 구성에서만 측정했습니다.
장시간 운전	8시간 연속 운전 시험은 하지 않았습니다.
수락 시험 묶음	T01~T12를 17개로 나눠 통과했지만 한 번에 묶은 실행은 하지 않았습니다.
공개 배포 알람	알람 생성이 공개 배포에는 연결되지 않았습니다(규칙 0개).
결과 불명 1건	촬영 중 설정 변경 1건이 결과 불명으로 남았고 원인은 조사하지 않았습니다.
공개 데모 운영	무료 DB 한도로 프로젝트를 한 번 옮겼고, 수집 스트림을 한 번 초기화했습니다.

자료

GitHub 저장소 ↗

공개 데모 ↗

시스템 기획서 ↗

개발계획서 ↗

명령·사건 계약 ↗

개발 진행 기록 ↗

수락 시험 대응표 ↗

사용자 매뉴얼 ↗

자료 범위

화면: 공개 배포 · 2026-10-29 실제 운전

수치: 로컬 PostgreSQL 마이그레이션 적용 결과와 2026-10-23 커밋 c8a24c1 로컬 전체 검사

코드: 고정 커밋 8134cb8

작성자 전우석 · 개발 기간 2026.10.20 ~ 10.28 (9일)

계정 비밀번호·DB 접속정보·인증서는 포함하지 않았습니다.